

Practical 7

* Aim:- Implement a stack using arrays in C++, and perform push, pop, and display operations.

* Theory:- A stack is a linear data structure that follows the LIFO (Last in, First out) principle. This means the element inserted last is removed first, just like stacking plates - only the top plate can be taken off. The size of the stack is fixed using a constant ($MAX=5$). An integer variable `top` is used to keep track of the topmost element of the stack. Initially, `top` is set to -1 , which indicates that the stack is empty.

Basic Stack Operations:-

① Push Operation:-

(a) The push operation is used to insert an element into the stack.

(b) Before inserting, the program checks whether the stack is full.

(c) If $top == MAX-1$, it means the stack is full and stack overflow occurs.

(d) Otherwise, `top` is incremented and the value is stored at `stack[top]`.

② Pop operation:-

(a) The pop operation removes the topmost element from the stack.

(b) Before removing, the program checks whether

the stack is empty.

③ If $top == -1$, stack underflow occurs.

④ Otherwise, the element at $stack[top]$ is displayed and top is decremented.

③ Display Operation:-

① This operation displays all elements of the stack from top to bottom.

② If the stack is empty ($top == -1$), an appropriate message is shown.

③ Otherwise, elements are printed using a loop from top to 0.

* Menu-Driven Program

The program uses a menu-driven approach that allows the user to:-

(i) Push element into the stack.

(ii) Pop elements from the stack.

(iii) Display stack elements.

(iv) Exit the program.

This makes the program interactive and easy to understand.

* Conclusion:- In this practical, a stack was implemented using an array in C++. The basic operations such as push, pop and display were performed successfully by following the LIFO principle. The program also handled stack overflow and underflow conditions properly.